@tetherto/wdk-protocol-fiat-transak

> This package is in beta. Please test in a dev setup first.

Builds Transak widget URLs and quotes for on-ramp (buy) and off-ramp (sell) flows. Works in both frontend and backend code.

Usage

Use the guides below for setup, trading, and transaction follow-up.

Guide	What it covers
Get Started	Install the package and initialize `TransakProtocol`.
Buy and Sell	On-ramp, off-ramp, quotes, supported assets, widget options, recipients.
Manage Transactions	Check status and load transaction details from Transak.
Configuration	API keys, caching, and Transak configuration options.
API Reference	Constructor, methods, and types for `TransakProtocol`.
Node.js Quickstart	Get started with WDK in a Node.js environment.

About WDK

Part of WDK (Wallet Development Kit) — tools for building safe, non-custodial wallets. Read more at https://docs.wallet.tether.io.

Get Started

Installation

```
npm install @tetherto/wdk-protocol-fiat-transak
```

Initialize TransakProtocol

Create an instance with an optional wallet account and a config object:

```
import TransakProtocol from '@tetherto/wdk-protocol-fiat-transak'

const transak = new TransakProtocol(account, {
  apiKey: 'YOUR_TRANSAK_PARTNER_KEY',
  widgetUrl,               // required for buy()/sell() — see below
  environment: 'STAGING'   // 'PRODUCTION' (default) | 'STAGING'
})
```

- **`account`** — an optional WDK wallet account (`IWalletAccount` / `IWalletAccountReadOnly`), or `undefined`. Used only by `buy`/`sell` to auto-fill the wallet address (see Buy and Sell).
- **`config`** — see Configuration for all options.

`widgetUrl` is a callback that turns the assembled `widgetParams` into a widget URL. It runs on **your backend**, where your API secret is safe:

```
const widgetUrl = async (widgetParams) => {
  const res = await fetch('https://your-backend.example.com/transak/widget-url', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ widgetParams })
  })
  if (!res.ok) throw new Error(`Failed to create Transak widget URL: ${res.status}`)
  const { widgetUrl } = await res.json()
  return widgetUrl
}
```

See Creating the widget URL (backend) for what that backend endpoint does.

Buy and Sell

`buy`/`sell` return a Transak widget URL; `quoteBuy`/`quoteSell` preview the economics without opening the widget.

Amounts are in **base units** (`number | bigint`) — fiat in cents (`10_000n` = $100), crypto in on-chain base units (`1_000_000_000_000_000_000n` = 1 ETH). Quotes return the same way (`bigint` base units; `rate` a decimal string).

```
// Buy (on-ramp)
const { buyUrl } = await transak.buy({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
  fiatAmount: 10_000n,   // $100 in cents — or pass cryptoAmount
  recipient: '0xabc'     // optional; falls back to the bound account
})

// Sell (off-ramp)
const { sellUrl } = await transak.sell({
  cryptoAsset: 'ETH',
  fiatCurrency: 'USD',
```

```
  cryptoAmount: 100_000_000_000_000_000n, // 0.1 ETH in wei
  refundAddress: '0xabc'                  // optional
})

// Quote (no widget opened)
const quote = await transak.quoteBuy({ cryptoAsset: 'ETH', fiatCurrency: 'USD', fiatAmount: 1
// → { cryptoAmount, fiatAmount, fee (bigint base units), rate (string), metadata }
```

Provider-specific extras — `paymentMethod`, `network`, and other Transak widget parameters — go under `config`:

```
await transak.buy({
  cryptoAsset: 'USDT', fiatCurrency: 'USD', fiatAmount: 10_000n,
  config: { network: 'tron', paymentMethod: 'credit_debit_card' }
})
```

Notes

- **Conventions** — `cryptoAsset`/`fiatCurrency` are upper-case (`ETH`, `USD`), `network`/`paymentMethod` lower-case (`ethereum`, `credit_debit_card`). They're matched exactly, with no normalisation (a wrong case throws `Cannot find info for cryptoAsset and fiatCurrency`). Fetch the exact values with the supported-currencies methods.
- **Multi-network assets** — a symbol like `USDT` exists on several chains; `config.network` picks one (first match if omitted).
- **Wallet address** — resolved from `recipient`/`refundAddress`, else the bound account's address, else the Transak widget prompts the user. `quote*`, `getTransactionDetail`, and `getSupported*` never use the account.

Manage Transactions

Get transaction details

Look up an order by its Transak ID:

```
const detail = await transak.getTransactionDetail('order-id')
// {
//   status: 'completed',        // 'in_progress' | 'failed' | 'completed'
//   cryptoAsset: 'ETH',
//   fiatCurrency: 'USD',
//   metadata: { id: 'order-id', status: 'COMPLETED', /* …full Transak order */ }
// }
```

`status` normalises Transak's order status to a standard value; the raw Transak order is under `metadata`.

Supported currencies and countries

Fetch the exact values your account supports at runtime (cached per `cacheTime`). Use `code`/`networkCode` verbatim as `cryptoAsset`/`network`, and `code` as `fiatCurrency`; `paymentMethod` ids are in `metadata.paymentOptions[].id`.

```
await transak.getSupportedCryptoAssets()
// [{ code: 'ETH', networkCode: 'ethereum', decimals: 18, name, metadata }, …]

await transak.getSupportedFiatCurrencies()
// [{ code: 'USD', decimals: 2, name, metadata }, …]

await transak.getSupportedCountries()
// [{ code: 'US', isBuyAllowed: true, isSellAllowed: true, name, metadata }, …]
```

Configuration

`new TransakProtocol(account, config)` — the `config` object:

Option	Type	Default	Description
`apiKey`	string	—	Your Transak partner API key (Partner dashboard).
`widgetUrl`	function	—	Callback that receives the assembled `widgetParams` object and returns a widget URL. **Required for `buy`/`sell`** (they throw without it); `quoteBuy`, `quoteSell`, and the `getSupported*` methods don't need it.
`environment`	`'PRODUCTION'` | `'STAGING'`	`'PRODUCTION'`	Selects the Transak API host. Use `'STAGING'` for testing with non-real funds.
`cacheTime`	number	`600000`	Milliseconds to cache the supported crypto/fiat lists.

API Reference

Method	Description	Returns
`buy(options)`	Generates a widget URL to purchase crypto	`Promise<{ buyUrl }>`
`sell(options)`	Generates a widget URL to sell crypto	`Promise<{ sellUrl }>`

`quoteBuy(options)`	Gets a quote for a crypto asset purchase	`Promise<TransakBuyQuote>`
`quoteSell(options)`	Gets a quote for a crypto asset sale	`Promise<TransakSellQuote>`
`getTransactionDetail(txId)`	Retrieves the details of an order	`Promise<TransakTransactionDetail>`
`getSupportedCryptoAssets()`	Lists supported crypto assets	`Promise<TransakSupportedCryptoAsset[]>`
`getSupportedFiatCurrencies()`	Lists supported fiat currencies	`Promise<TransakSupportedFiatCurrency[]>`
`getSupportedCountries()`	Lists supported countries	`Promise<TransakSupportedCountry[]>`

`buy(options)` / `sell(options)`

- `cryptoAsset` (string): The crypto asset code (e.g. `'ETH'`).
- `fiatCurrency` (string): The fiat currency code (e.g. `'USD'`).
- `fiatAmount` (number | bigint, optional): The fiat amount, in base units (e.g. `10_000n` = 100 USD in cents).
- `cryptoAmount` (number | bigint, optional): The crypto amount, in base units (e.g. `1_000_000_000_000_000_000n` = 1 ETH in wei).
- `recipient` (string, optional, `buy` only): The destination wallet address. Falls back to the account address.
- `refundAddress` (string, optional, `sell` only): The refund wallet address. Falls back to the account address.
- `config` (object, optional): Additional Transak widget parameters, including `network`.

Returns `Promise<{ buyUrl }>` / `Promise<{ sellUrl }>` — the URL your `widgetUrl` callback produced, e.g. `https://global.transak.com/?apiKey=…&sessionId=…`.

`quoteBuy(options)` / `quoteSell(options)`

- `cryptoAsset` (string): The crypto asset code.
- `fiatCurrency` (string): The fiat currency code.
- `fiatAmount` (number | bigint): The fiat amount in base units. `quoteBuy` accepts this or `cryptoAmount`.
- `cryptoAmount` (number | bigint): The crypto amount in base units. Required for `quoteSell`; `quoteBuy` accepts this or `fiatAmount`.
- `config` (object, optional): Provider-specific extras — `paymentMethod` and `network` (resolved from the supported list when omitted).

Returns `Promise<TransakBuyQuote>` / `Promise<TransakSellQuote>`:

```
{
  cryptoAmount: 500000000000000000n, // base units (wei)
  fiatAmount: 100000n,               // base units (cents)
  fee: 500n,                         // base units (cents)
  rate: '2000',                      // decimal string, standard units
  metadata: { quoteId: 'q1', conversionPrice: 2000, /* …full Transak quote */ }
}
```

`getTransactionDetail(txId)`

- `txId` (string): The Transak order ID.

Returns `Promise<TransakTransactionDetail>` — `{ status, cryptoAsset, fiatCurrency, metadata }`, where `status` is `'in_progress' | 'failed' | 'completed'`.

`getSupportedCryptoAssets()` / `getSupportedFiatCurrencies()` / `getSupportedCountries()`

Return arrays of, respectively:

- `{ code, networkCode, decimals, name, metadata }`
- `{ code, decimals, name, metadata }`
- `{ code, isBuyAllowed, isSellAllowed, name, metadata }`

The raw Transak object is always available under `metadata`.

Creating the widget URL (backend)

`buy` and `sell` build a `widgetParams` object and hand it to your `widgetUrl` callback. That callback runs on your backend, where your API secret is safe, and turns the params into a widget URL using Transak's API-based flow (passing params directly in the URL is deprecated). It's two calls:

```
// On your backend — never ship the API secret to the client.
async function createWidgetUrl (widgetParams) {
  // 1. Get a partner access token (cache it until it expires).
  const tokenRes = await fetch('https://api.transak.com/partners/api/v2/refresh-token', {
    method: 'POST',
    headers: { 'api-secret': process.env.TRANSAK_API_SECRET, 'content-type': 'application/jso
    body: JSON.stringify({ apiKey: process.env.TRANSAK_API_KEY })
  })
  const { data: { accessToken } } = await tokenRes.json()

  // 2. Create the widget session and return its URL.
  const sessionRes = await fetch('https://api-gateway.transak.com/api/v2/auth/session', {
    method: 'POST',
    headers: { 'access-token': accessToken, 'content-type': 'application/json' },
    body: JSON.stringify({ widgetParams })
  })
  const { data: { widgetUrl } } = await sessionRes.json()
  return widgetUrl // valid for 5 minutes, single use
}
```

Notes:

- Keep the API secret on the backend, never in client code.
- The returned widget URL is valid for **5 minutes** and each session can be used **once** — create a fresh one per flow.
- For staging, use `https://api-stg.transak.com` and `https://api-gateway-stg.transak.com`.
- For the full list of widget parameters, see the Transak query parameters docs.

Development

```
npm install            # install dependencies
npm run build:types    # build TypeScript definitions
npm run lint           # lint code
npm run lint:fix       # fix linting issues
npm test               # run tests
npm run test:coverage  # run tests with coverage
```

License

Apache License 2.0 — see the LICENSE file for details.

Contributing

Contributions are welcome! Please feel free to submit a Pull Request.

Support

For support, please open an issue on the GitHub repository.